

Programmation Orientée Objet (POO)

1. Introduction

Python est un langage fondamentalement orienté objet. **Tout est objet**, en Python. Dès la classe de première, vous avez *utilisé* la **notation pointée**, caractéristique de la POO :

```
lst = [10]      # lst est un OBJET de type <list>
lst.append(20)  # append() est une MÉTHODE de l'objet « lst »
```

En terminale, on apprend à **créer de nouveaux objets**, adaptés à la résolution d'un problème spécifique.

1.1. Idée générale

En programmation **procédurale**, les données et les fonctions qui les manipulent sont séparées. Pour un petit programme, c'est suffisant. Dès que le projet grossit, on perd le fil : quelle variable modélise quelle « chose » ? évolue selon quelle fonction ? Qui a le droit de la modifier ?

La **programmation orientée objet (POO)** répond à ce problème en regroupant **données** et **comportements** dans une même entité : l'**objet**.

1.2. L'analogie du moule à gâteaux

Il faut imaginer un **moule à gâteaux** : le moule lui-même **n'est pas** un gâteau, mais il définit la forme que *tous* les gâteaux produits auront. Chaque gâteau sorti du moule est un exemplaire concret, avec ses propres caractéristiques.

- Le **moule** ↔ la **classe** (le plan, le modèle) ;
- Un **gâteau précis** ↔ un **objet** (une instance) ;
- La **saveur, la taille** ↔ les **attributs** (l'état propre à chaque objet) ;
- Ce qu'on peut **faire avec** (couper, décorer) ↔ les **méthodes** (les comportements).

⚠ Limites de l'analogie

Un moule est *passif* : il ne « fait » rien. En POO, la **classe** contient du code actif (les méthodes). De plus, un moule impose une forme physique rigide alors qu'une classe peut définir des comportements très divers.

Ne pas confondre non plus : le moule ne *contient* pas les gâteaux — une classe ne contient pas ses instances en mémoire au sens « physique ».

2. Définitions

Classe : type abstrait de données défini par le programmeur, regroupant :

- des **attributs** : variables qui décrivent un *état* ;
- des **méthodes** : fonctions qui définissent un « comportement ».

Objet (instance) : exemplaire concret créé à partir d'une classe. Créer un objet s'appelle **instancier** la classe. On abrège « objet instance d'une classe » en « instance d'une classe ». Chaque objet (ou instance) créé à partir d'une classe possède ses *propres* valeurs d'attributs, et peut invoquer les méthodes de sa classe.

Constructeur : méthode spéciale `__init__(self, ...)` appelée automatiquement à l'instanciation. Elle initialise les attributs de l'objet.

 **Remarque : en réalité...** ▼

Pour la culture : `__init__(self, ...)` n'est en réalité pas le constructeur, mais l'*initialiseur* (la méthode chargée de créer un objet avec « les bons attributs »). En NSI, on assimile `__init__(self, ...)` à `__new__(...)`, qui est le *vrai* constructeur.

self : premier paramètre conventionnel de *toute méthode* d'instance. Il désigne l'instance courante — c'est le mécanisme par lequel une méthode sait *sur quel objet* elle agit.

Encapsulation : principe consistant à regrouper attributs et méthodes dans la classe, et à contrôler l'accès aux données internes en distinguant interface publique et implémentation interne.

Agrégation : une classe peut avoir comme attributs des instances d'autres classes. On parle de relation « *a-un* » : un objet « *contient* » d'autres objets.

 **Remarque : classe ou type** ▼

Ces deux notions sont quasiment synonymes : une classe *est* un type — un type prédéfini en Python (pour les types natifs), ou que le programmeur définit lui-même pour ses propres besoins, à l'aide de la syntaxe exposée ci-dessous. Afficher le type d'une variable avec `type()` fait directement référence à une classe (on voit explicitement le mot `class`).

- `int`, `str`, `list`, `dict` ... sont des classes natives de Python, donc des types.
- Quand on définit une classe avec `class MaClasse: ...`, on crée un *nouveau type*, « vivant » aussi longtemps que le programme qui l'a défini tourne.
- `type(objet)` renvoie toujours la classe dont est issue l'objet.

3. Un exemple en Python

Cet exemple est inspiré par un sujet de bac NSI ([2026, Amérique du Nord J2, exercice 2](#)).

3.1. Définition et utilisation basiques

Une classe Python se définit par le mot-clef `class`. Il est suivi par le nom de la classe. Il est d'usage que ce nom commence par une majuscule, et recoure à la notation « *PascalCase* » si nécessaire.

Classe Python

```
class Joueuse:
    def __init__(self, nom, prenom, pays, age, point):
        # Attributs publics (accès et modification libres depuis l'extérieur)
        self.nom = nom
        self.prenom = prenom
        self.pays = pays
        self.age = age
        self.point = point                # Points WTA (tournoi mondial féminin)
        self.victoire = 0
        self.defaite = 0

    def ajouter_victoire(self, adversaire):    # self désigne l'objet lui-même
        self.victoire += 1
        adversaire.defaite += 1
```

Instanciation d'objets et accès aux attributs

À partir de la classe `Joueur`, on crée deux objets pour représenter deux joueuses de tennis :

```
swiatek = Joueuse("Swiatek", "Iga", "POL", 22, 7470)
sabalenka = Joueuse("Sabalenka", "Aryna", "BLR", 25, 10541)
```

Chaque variable (`swiatek` ou `sabalenka`) désigne alors un objet *spécifique* et *autonome*. Dans ce contexte, on associe les termes *variable* et *objet* : on peut employer l'un ou l'autre indifféremment.

Chaque *variable* / *objet* dispose des *mêmes noms* d'attribut, par exemple `prenom`. Mais la **valeur** de l'attribut `prenom` de l'objet `swiatek` **DIFFÈRE** de celle de l'attribut `prenom` de l'objet `sabalenka`.

✓ Accéder à un attribut

La **notation pointée** permet d'accéder à un attribut spécifique d'un objet : `objet.attribut`.

Exemple : ▼

```
print(swiatek.prenom)        # Affiche « Iga »
print(sabalenka.prenom)      # Affiche « Aryna »
```

Utilisation d'une méthode

La notation pointée permet également d'appeler une méthode d'un objet. Le paramètre `self` est une référence à l'objet lui-même, au sens « l'objet *précis* qui appelle cette méthode ». **IL NE faut JAMAIS l'indiquer lors de l'APPEL d'une méthode** : Python le passe *automatiquement* à la méthode. Donc, lors de l'**appel** d'une méthode, il suffit de passer les

paramètres en nombre (et en type) adapté *sans tenir compte de* `self`.

Concrètement, une victoire de Swiatek sur Sabalenka se traduirait en Python par :

```
swiatek.ajouter_victoire(sabalenka)
```

La méthode `ajouter_victoire`, définie dans la classe `Joueuse`, sait qu'elle « travaille depuis » l'objet `swiatek` car son paramètre `self` désigne *automatiquement* `swiatek`. De même, cette méthode sait que c'est le compteur de victoire de `swiatek` (donc `swiatek.victoire`) qui doit être incrémenté : le `self` de `self.victoire += 1` fait là encore référence, dans ce contexte, à l'objet `swiatek`. Donc `self.victoire` signifie ici, en réalité, `swiatek.victoire`.

Remarque : accès aux attributs d'un autre objet ▼

Le paramètre `adversaire` de la méthode `ajouter_victoire` est (implicitement) censé être une instance de la classe `Joueuse`. Python accède alors à son attribut `defaite` et l'incrmente. Ainsi, dans l'instruction `swiatek.ajouter_victoire(sabalenka)`, lors de l'exécution `adversaire.defaite += 1`, la méthode `ajouter_victoire` a reçu comme valeur `sabalenka` pour le paramètre `adversaire`. C'est donc bien l'attribut `defaite` de l'objet `sabalenka` qui est incrémenté.

3.2. Pour la culture : méthodes standards de Python

TOUTE cette partie est hors programme : s'il est inutile de savoir *faire*, mieux vaut savoir que *ça existe* (un sujet de bac au moins a superficiellement abordé ce thème).

Python prévoit, pour chaque objet, des méthodes aux noms standards qui permettent certaines facilités dans l'utilisation du langage.

Ainsi, la commande `print(swiatek)` affiche en réalité la valeur de retour de la méthode `__str__()` de l'objet `swiatek`. Concrètement, `print(swiatek)` est (à peu près) équivalent à `print(swiatek.__str__())` (ce qui est moins parlant et moins pratique). On parle de *sucre syntaxique* (le sucre c'est doux, c'est agréable).

Cette méthode possède une définition *par défaut*, qu'on peut « écraser » par un code plus adapté aux besoins : on parle de *surcharge* de la méthode.

La méthode `__lt__(self, autre)` (inexistante par défaut) permet, lorsqu'elle est définie, d'utiliser l'opérateur de comparaison `<` entre deux instances de la classe `Joueuse` (`lt` signifie *less than*).

Pour cela, il suffit d'ajouter à la classe `Joueuse` le code ci-dessous (par exemple).

```
def __str__(self):
    # Permet un affichage adapté lorsqu'on écrit `print(j1)` (`j1` est une instance de `Joueuse`)
    return f"{self.prenom} {self.nom} ({self.pays}) – {self.point} pts WTA"

def __lt__(self, autre):
    # Permet d'écrire : j1 < j2 (vrai si j1 a moins de points WTA que j2)
    return self.point < autre.point
```

Utilisation

```
print(swiatek)           # Affiche « Iga Swiatek (POL) – 7470 pts WTA »
print(swiatek < sabalenka) # Affiche « True », car 7470 < 10541
```

 **Remarque : un code « plus propre » grâce aux accesseurs et mutateurs ▼**

Pour la culture

Le fait d'accéder « depuis l'extérieur » (ici l'objet `swiatek`) à l'attribut `defaite` de l'objet `sabalenka` est permis par Python. Ça n'est pas « propre » pour autant : accéder directement à un attribut est souvent considéré comme une *mauvaise pratique*, spécialement quand il s'agit de le *modifier*. En effet, il peut être souhaitable d'effectuer des vérifications (sur le domaine des valeurs admissibles, par exemple) : en modifiant *directement* la valeur, on contourne ces (éventuelles) vérifications. Une *meilleure* pratique consiste à utiliser des méthodes pour accéder aux attributs, en lecture aussi bien qu'en écriture : on les appelle **accesseurs** (pour l'accès) et **mutateurs** (pour la modification).

Voici une illustration de ce principe, menant à une version *plus « propre »* du code précédent. La méthode `ajouter_victoire` est un mutateur pour l'attribut `victoire`. On ajoute la méthode `ajouter_defaite`, qui est son pendant pour l'attribut `defaite`. La méthode `ajouter_victoire`, au lieu de modifier *directement* l'attribut `defaite` d'un autre objet, demande par l'intermédiaire de la méthode `ajouter_defaite` de cet autre objet d'incrémenter son attribut `defaite`.

```
def ajouter_victoire(self, adversaire):    # self désigne l'objet lui-même
    self.victoire += 1
    adversaire.ajouter_defaite()

def ajouter_defaite(self):
    self.defaite += 1
```

En Python, lorsque l'on veut **signifier** à un autre développeur (qui voudrait utiliser notre code) qu'il faut éviter de modifier directement un attribut, on le préfixe par `_`. Ainsi, un attribut nommé `_gains` indique par son seul nom qu'il **ne** faut **pas** le modifier depuis l'extérieur (pas de `objet._gains = 1000`, c'est la convention « ne pas toucher »). Dans ce cas, il *faut* fournir un **accesseur** (on dit *getter* en anglais) et un **mutateur** (*setter*).

Si l'on veut se montrer *plus décourageant* encore, on peut préfixer un attribut qu'on veut rendre aussi « privé » que possible avec un double *underscore* : `self.__pas_touche`.

Python n'est pas Java

En Python, **rien n'est vraiment privé**. Le préfixe `_` est une *convention* : elle signale « ne pas accéder directement », mais n'interdit techniquement rien. Le préfixe `__` déclenche un *name mangling* (`__x` devient `_NomClasse__x`), ce qui complique l'accès depuis l'extérieur sans l'interdire. C'est une « barrière sociale », pas une « serrure ».

En Java ou en C++, on doit explicitement distinguer les attributs qu'on peut lire ou écrire depuis l'extérieur d'un objet de ceux pour lesquels c'est impossible. Et c'est *obligatoire*.

3.3. Agrégation : la classe Match

La classe ci-dessous modélise un match entre deux joueuses. L'instanciation d'un objet de classe `Match` **nécessite** l'existence préalable de *deux objets* de classe `Joueuse`, qui sont passés en paramètres au constructeur de la classe `Match`. C'est ce qu'on appelle l'**agrégation** : on agrège ces deux objets au sein d'un troisième.

La méthode `resultat_match` permet d'enregistrer dans l'objet le score d'un match et d'en déterminer l'issue. *En bonus*, la redéfinition d'une méthode standard permet d'en afficher facilement l'issue.

```
class Match:
    def __init__(self, intitule, joueuse1, joueuse2):
        self.intitule = intitule
        self.joueuse1 = joueuse1          # objet de classe Joueuse
        self.joueuse2 = joueuse2          # objet de classe Joueuse
        self.gagnante = None
        self.perdante = None
        self.score = None

    def resultat_match(self, score):
        # score : liste de tuples (jeux_j1, jeux_j2) par set
        self.score = score
        nb_sets_j1 = sum(1 for s in score if s[0] > s[1])
        nb_sets_j2 = sum(1 for s in score if s[1] > s[0])
        if nb_sets_j1 > nb_sets_j2:
            self.gagnante = self.joueuse1
            self.perdante = self.joueuse2
        else:
            self.gagnante = self.joueuse2
            self.perdante = self.joueuse1
        self.gagnante.ajouter_victoire(self.perdante)

    def __str__(self):
        if self.gagnante is None:
            return f"{self.intitule} : résultat inconnu"
        return f"{self.intitule} : victoire de {self.gagnante.prenom} {self.gagnante.nom}"
```

Utilisation :

```
finale = Match("Finale Madrid 2024", swiatek, sabalenka)
finale.resultat_match([(7, 5), (4, 6), (7, 6)])
print(finale)          # Affiche « Finale Madrid 2024 : victoire de Iga Swiatek »
```

Remarque : agrégation vs simple variable ▼

`self.joueuse1` ne stocke pas une *copie* de l'objet `Joueuse` : il stocke une **référence** vers l'objet existant. Dans l'exemple d'utilisation ci-dessus, modifier `self.joueuse1.victoire` modifie bien l'objet original `swiatek`.

4. Points de vigilance et exigences du bac

⚠ Erreurs classiques

- **Oublier `self`** comme premier paramètre d'une méthode → `TypeError` cryptique du type « *takes 1 positional argument but 2 were given* ».
- **Écrire `valeur = valeur`** au lieu de `self.valeur = valeur` dans `__init__` → l'attribut n'est pas créé, juste une variable locale *immédiatement perdue*.
- **Confondre classe et objet** : `Joueuse` est la classe ; `swiatek = Joueuse(...)` est un objet. Appeler `Joueuse.ajouter_victoire()` sans instance provoque une erreur.
- **Appeler `__init__` manuellement** : `obj.__init__(...)` réinitialise un objet déjà existant. Cela fonctionne, mais signe presque toujours une erreur de conception.
- **Confondre agrégation et copie** : `self.joueuse1 = joueuse1` stocke une *référence*, pas une copie. Modifier l'objet *via* un attribut le modifie partout où il est référencé.

✅ Récapitulatif : ce qu'il faut maîtriser pour le bac

- Lire, écrire et compléter une définition de classe avec `__init__`, attributs, `self`, méthodes.
- Instancier des objets et appeler leurs méthodes via la notation pointée.
- Comprendre et appliquer l'encapsulation (conventions `_`/`__`, getters/setters).
- Lire et utiliser `__str__` pour l'affichage.
- Comprendre l'agrégation (objet contenant d'autres objets) et l'interaction entre deux classes.
- **Lire et utiliser** une redéfinition d'opérateur de comparaison (`A < B` sur des objets), sans nécessairement l'écrire de mémoire.
- Lire et compléter une méthode récursive définie dans une classe, opérant sur une structure arborescente.

5. Liens avec d'autres notions

- **Structures de données** : les listes chaînées, piles, files, arbres et graphes vus en NSI sont très souvent implémentés via des classes. Un nœud d'arbre binaire est typiquement une classe avec attributs `valeur`, `gauche`, `droite`.
- **Modules Python** : une classe est souvent définie dans un fichier `.py` séparé et importée — c'est la base de la modularité logicielle.
- **Paradigmes de programmation** : la POO est l'un des trois paradigmes abordés en NSI, avec la programmation impérative/procédurale et la programmation fonctionnelle.
- **Récursivité** : les méthodes récursives sur des objets (voir dans le code ci-dessous la méthode `mise_a_jour`) suivent exactement le même schéma que les fonctions récursives ordinaires.

Méthode récursive sur une structure arborescente d'objets

La POO et la récursivité se combinent naturellement dès qu'une classe contient des attributs qui sont eux-mêmes des instances de la même classe (ou d'une classe liée). C'est le cas typique de la classe `Arbre` dont les nœuds sont des objets

Match (se reporter au sujet 26-NSIJ2AN1, exercice 2, partie B) :

```
class Arbre:
    def __init__(self, racine, gauche, droite):
        self.racine = racine                # Objet de classe Match
        self.gauche = gauche                # Objet de classe Arbre (ou None)
        self.droite = droite                # Objet de classe Arbre (ou None)

    def mise_a_jour(self):
        """Propage les gagnantes des sous-arbres vers la racine."""
        # Cas de base : joueuse1 du match-racine est déjà connue.
        # Dans ce cas, il n'y a rien à faire. Sinon...
        if self.racine.joueuse1 is None:
            if self.gauche is not None:      # S'il y a un vrai SAG
                # Appel récursif sur le sous-arbre gauche
                self.gauche.mise_a_jour()
                # Remontée : on récupère la gagnante du match à gauche
                self.racine.joueuse1 = self.gauche.racine.gagnante
        # Cas de base bis : joueuse2 du match-racine est déjà connue.
        # Dans ce cas, il n'y a rien à faire. Sinon...
        if self.racine.joueuse2 is None:
            if self.droite is not None:      # S'il y a un vrai SAD
                # Appel récursif sur le sous-arbre droit
                self.droite.mise_a_jour()
                self.racine.joueuse2 = self.droite.racine.gagnante
```

Schéma récursif :

- **cas de base** : la joueuse concernée est déjà connue dans la racine → rien à faire ;
- **appel récursif** : on descend dans le sous-arbre gauche (ou droit) pour y trouver l'objet `Joueuse` gagnante ;
- **remontée** : on récupère la gagnante du « sous-match » (le match du tour précédent) pour l'injecter dans le match courant.

Remarque : lisibilité ? ▼

L'instruction `self.racine.joueuse1 = self.gauche.racine.gagnante` est délicate à appréhender. Elle exige en effet de garder en tête simultanément trois « niveaux » de types *imbriqués*.

- `self.gauche` finit toujours par être un objet de la classe `Arbre` (ou `None`, au début).
- `self.gauche.racine` est un objet de la classe `Match`.
- `self.gauche.racine.gagnante` est un objet de la classe `Joueuse`.

Se souvenir de tout cela n'a rien d'évident. Dans ce genre de situation, mieux vaudrait avoir des attributs portant des noms plus explicites, quitte à avoir un code plus verbeux. Cela donnerait par exemple

```
self.tableau_gauche.match_racine.joueuse_gagnante.
```